Parapet: Technical Documentation

2026 Shenzhen International FinTechathon, International Track

Topic E: SME Finance Copilot

Team: Oneshot (Bendi Tandayu Saputra, Made Kevin Darmawan, Aqsha Lathif Imron, Nawal Alhamid)

Repository: https://github.com/benditandayusaputra/parapet

Version: 0.1.0, 20 October 2026

---

1. Summary

Parapet is an AI finance agent for small and medium-sized businesses that acts as a financial participant on the owner's behalf in both directions of cash flow:

- It **collects what the business is owed** by negotiating settlement terms with the buyer's agent, locking those terms with a signature, and escalating to invoice financing when a high-risk customer will not commit.
- It **refuses to pay what the business does not owe** by scoring every supplier invoice for fraud signals and, whenever payee bank details are new or changed, running a cryptographic verification handshake with the supplier's agent before any money moves.

The two directions share one engine: the Counterparty Verification Protocol, a four-level permission-tier policy set by the owner, and an append-only hash-chained audit log. The language model never makes a decision; it only explains a decision that the deterministic policy engine has already made.

2. Problem

Small businesses lose money in two places, and both are invisible until it is too late.

Money in. In Hong Kong, 96% of SMEs reported a cash-flow crisis in the past 12 months, and nearly three in five owners spend up to five hours a month chasing overdue invoices (Airwallex SME study, 2026). Across APAC, 57% of firms reported late payments and 52% expect conditions to worsen (Coface APAC Payment Survey, 2026). Bank of Thailand data puts SME loan rejection at roughly 60 to 70%, so borrowing to bridge late receivables is often not an option.

Money out. The OECD Consumer Finance Risk Monitor 2026, covering 60 jurisdictions, ranks scams and fraud as the number one financial risk, with the most common pattern being a fraudster posing as a known counterparty. For businesses this takes the form of business email

compromise: a supplier's mailbox is hijacked, an invoice arrives with "updated bank details", the finance person pays, and the money is gone. SMEs have no finance department to catch it.

Existing tools treat these as separate problems: invoice-reminder software on one side, fraud filters on the other. Both are passive. Neither verifies the counterparty, and neither acts.

3. Solution overview

Parapet is one agent with two procedures and one protocol.

```
supplier invoice --> process_payable()
                     1. risk.score_payable
                     2. CVP payee attestation  <--> supplier's agent
                     3. policy.resolve
                     4. ledger.execute (only if EXECUTED)
                     --> pay / prepare / hold / escalate

overdue invoice  --> process_receivable()
                     1. risk.score_receivable
                     2. CVP settlement rounds   <--> buyer's agent
                     3. policy tier applied
                     --> agreed terms / follow-up / financing

both paths       --> hash-chained audit log, owner message
```

The user-facing promise is one sentence: *Parapet collects what you are owed and refuses to pay what you do not owe.*

4. System architecture

4.1 Components

Module	Responsibility
`models.py`	Typed data model: `Invoice`, `Counterparty`, `BankAccount`, `Policy`, `Tier`, `Decision`, `RiskReport`
`risk.py`	Explainable additive risk scoring for payables and receivables
`handshake.py`	Counterparty Verification Protocol: canonical payloads, HMAC signing, challenge and proposal types
`counterparties.py`	Counterparty agents (legitimate supplier, impostor, buyer) used in the sandbox
`policy.py`	Permission-tier engine: maps (tier, amount, risk, handshake result, spend today) to a decision
`agent.py`	`ParapetAgent` orchestration: `process_payable`,

Module	Responsibility
	`process_receivable`
`audit.py`	Append-only, SHA-256 hash-chained audit log with tamper detection
`ledger.py`	Sandbox ledger (balance, invoices, transfers) and seed data
`reasoner.py`	Plain-language explanation for the owner: template by default, optional LLM
`cli.py`	Scenario runner producing execution evidence

4.2 Data flow for a payable

1. Invoice is received with amount, payout account, sender domain and message body.
2. `risk.score_payable` compares the invoice against the counterparty record and payment history and emits named signals with weights (section 5.1).
3. If the payout account is new or differs from the registered account, the agent issues a `PayeeChallenge` to the supplier's agent and verifies the returned signature against the secret exchanged at onboarding.
4. `policy.resolve` turns tier, amount, risk, caps and handshake result into one of five decisions.
5. Only `EXECUTED` touches the ledger. Every step writes to the audit log before and after.
6. `reasoner.explain` produces the owner message from the decision and the signals.

4.3 Data flow for a receivable

1. Overdue invoice is reviewed; `risk.score_receivable` scores default propensity from days overdue and the customer's on-time history.
2. The agent sends a `SettlementProposal` (full amount, pay within 3 days) to the buyer's agent.
3. Up to three rounds: the buyer accepts, or counters with a date. The agent accepts any counter within its 14-day ceiling; otherwise it concedes halfway and proposes again.
4. An agreement is only recorded when the buyer's signature over the agreed payload verifies.
5. With no agreement and risk at or above the owner's threshold, the agent escalates with an invoice-financing recommendation for that specific receivable.

4.4 Deployment architecture (target)

The sandbox ledger is the only component that touches money and is designed to be swapped for a bank's core APIs behind the same interface:

- **Balance and transfer:** bank core ledger.
- **Payee registry and KYB:** the bank's verified-payee records become the source of `registered_account` and the issuer of the handshake key, turning CVP into a bank-backed verified-payee check.

- **Counterparty agents:** suppliers and buyers run their own Parapet instance, or the bank hosts a lightweight attestation endpoint on their behalf.
- **Owner channel:** chat (WhatsApp, WeChat) or app notifications for tier-2 approvals and tier-0 alerts.

Target production stack: Go service for the policy and ledger adapter, Python for the risk and reasoning layer, PostgreSQL with append-only audit tables, message bus between the two.

5. Core algorithms

5.1 Explainable risk scoring

Scores are sums of named signal weights, capped at 1.0. Every score in the audit log is accompanied by its signals, so a reviewer can see exactly why a decision was made. There is no opaque model in the decision path.

Payable signals

Signal	Weight	Trigger
`NEW_PAYEE`	0.35	No prior relationship
`ACCOUNT_CHANGED`	0.45	Payout account differs from registered account
`DOMAIN_MISMATCH`	0.25	Sender domain differs from registered domain
`YOUNG_RELATIONSHIP`	0.10	Relationship under 30 days
`AMOUNT_ANOMALY`	0.15	Amount above 2x historical average for this payee
`URGENCY_LANGUAGE`	0.10 per phrase, max 0.20	“urgent”, “immediately”, “today”, “do not call”, “confidential”
`ACCOUNT_CHANGE_LANGUAGE`	0.10	Message announces new banking details

Receivable signals

Signal	Weight	Trigger
`OVERDUE`	0.05 per week overdue, max 0.50	Past due date
`POOR_HISTORY`	0.25	On-time ratio below 70%
`MIXED_HISTORY`	0.10	On-time ratio between 70% and 90%

Weights were set by hand for the sandbox and are configuration, not code. In production they are calibrated against the bank’s historical fraud and default labels.

5.2 Counterparty Verification Protocol (CVP)

CVP is a challenge-response protocol over a canonical JSON payload signed with HMAC-SHA256.

Payee attestation (payables). Challenge payload: `{type, invoice_id, amount, currency, bank, number, nonce}`. The supplier's agent signs the exact payload with the onboarding secret. The SME agent verifies with constant-time comparison. Three outcomes:

- Signature valid: payee verified. The invoice proceeds to policy resolution with the handshake flag set; residual risk signals can still route it to owner approval.
- Signature invalid or counterparty refuses: `HELD`. No money moves regardless of tier or amount.
- Counterparty unreachable: `HELD`. Absence of proof is treated as failure.

The key design property: an attacker who controls the supplier's email can reply "yes, confirmed" but cannot produce a valid signature, because the secret was never in the mailbox.

Settlement negotiation (receivables). Proposal payload: `{type, invoice_id, amount, currency, pay_by, nonce}`. The buyer agent accepts (signing the proposal) or counters (signing its counter). The agent only records terms whose signature verifies, so agreed terms are non-repudiable by the buyer.

Nonces prevent replay. In production the shared secret is replaced by an asymmetric key pair bound to the counterparty's verified business identity.

5.3 Permission-tier policy engine

```
resolve(policy, tier, amount, risk, spent_today, handshake_ok):
    if handshake_ok is False:          return HELD
    if risk >= hold_threshold:         return ESCALATED
    if tier == EXECUTE:
        if amount > per_tx_cap:        return PREPARED
        if spent_today + amount > daily_cap: return PREPARED
        return EXECUTED
    if tier == PREPARE:                return PREPARED
    if tier == RECOMMEND:              return RECOMMENDED
    return HELD
```

Order matters: verification failure and risk threshold are checked before tier, so no tier setting can override them. A verified handshake on a high-risk invoice downgrades `ESCALATED` to `PREPARED` (owner one-tap approval) rather than to `EXECUTED`.

5.4 Audit log

Each entry carries `seq`, UTC timestamp, event name, payload, `prev_hash` and its own SHA-256 `hash` over the canonical entry. `verify_chain()` recomputes the chain; the test suite demonstrates that modifying a single field breaks verification. The log is the execution evidence submitted with this entry.

6. Security design

Summarised here; the full permission-tier implementation and known-risk list are in `security_self_assessment.md`.

- **Least authority by construction.** The agent starts at tier 0 and the owner raises it. Caps are per transaction and per day. Tiers only reduce authority; they can never bypass a failed handshake or the risk threshold.
- **Model is not in the decision path.** The LLM produces text for the owner from an already-decided outcome. Prompt injection via invoice text cannot change a decision; it can only be scored as a risk signal.
- **Cryptographic verification, not conversational verification.** "Yes, confirmed" is worth nothing; a valid signature over the exact payment details is required.
- **Fail closed.** Unreachable counterparty, unknown payee, invalid signature all result in `HELD`.
- **Masked account numbers** in all logs and messages.
- **Tamper-evident audit trail** with a hash chain.

7. Interaction design

The owner's experience is deliberately small: three kinds of message.

- **"Done."** Tier-3 actions within caps, reported after the fact with the transfer reference.
- **"Ready for your approval."** One-tap confirmation with the reasons listed.
- **"Payment held" or "Needs you."** The agent explains the signals in plain language, states that no money has moved, and tells the owner exactly what to do next (for example, phone the supplier on the number on file, not the one in the email).

Every message includes the signal list, so the owner learns to recognise the patterns themselves over time.

8. Evidence of execution

Run `python -m parapet.cli --out evidence --today 2026-10-20`. The seeded sandbox produces five invoices and five distinct outcomes:

Invoice	Direction	Outcome	Why
INV-TINTA-0931	payable	Executed, TX-0001	Known account, within caps, risk 0.00
INV-ANEKA-2210	payable	Held, no money moved	Changed account, 2.1x amount, urgency language, impostor signature invalid; risk 0.90
INV-KEMAS-0788	payable	Prepared for owner	Changed account but supplier's agent signed correctly; risk

Invoice	Direction	Outcome	Why
			0.55
INV-OUT-5120	receivable	Agreed, pay by 2026-11-01	24 days overdue, poor history; buyer countered, agent accepted within ceiling
INV-OUT-5133	receivable	Agreed, pay by 2026-10-23	5 days overdue; buyer accepted first proposal

`evidence/audit_log.jsonl` contains the 31-entry hash-chained log of this run; `evidence/test_output.txt` shows the 12 passing tests.

9. Limitations and roadmap

- The sandbox uses a shared HMAC secret per counterparty; production requires asymmetric keys bound to KYB identity, ideally issued by the bank.
- Risk weights are hand-set; they need calibration on real fraud and default data.
- Counterparty agents are simulated; a real deployment needs an adoption path, which is why the bank-hosted attestation endpoint matters: a supplier does not need to run software for the bank to vouch for its account.
- Invoice ingestion (email, PDF parsing) is out of scope for the prototype.
- Financing escalation currently produces a recommendation; the next step is pre-filling the application against the bank's SME credit product.

10. Third-party components

Python standard library only for the sandbox. Optional: `anthropic` SDK (MIT) for model-written owner messages. No third-party data.